



UNIVERSITÀ DEGLI STUDI DI PADOVA

Corso di Laurea in Informatica
Insegnamento di Ingegneria del Software

Anno Accademico 2025/2026

Analisi dello Stato dell'Arte

Tecnologie e Soluzioni per la Realizzazione del Sistema

Gruppo: Synergo Unit

Email: synergounit.20@gmail.com

Versione: **v1.0.0**
Data: **2026-05-26**

Registro delle Modifiche

Versione	Data	Autore	Verificatore	Descrizione
1.0.0	2026-05-26	Roberto M. Doroftei		Approvazione per ingresso in baseline RTB.
0.1.0	2026-05-18	Francesco Marcon	Anton R. Rupi	Revisione del documento.
0.0.0	2026-05-13	Francesco Marcon	Anton R. Rupi	Prima stesura del documento.

Indice

1	Introduzione	3
1.1	Scopo del Documento	3
1.2	Scopo del Prodotto	3
1.3	Aree di Indagine	3
1.4	Glossario	3
1.5	Riferimenti	4
1.5.1	Normativi	4
1.5.2	Informativi	4
2	Tecnologie Frontend	5
3	Tecnologie Backend	5
4	Struttura Tecnica Preliminare del Proof of Concept	7
5	Distribuzione dell'Ambiente	7
6	Sicurezza e Protezione dei Dati	8
7	Scelte Finali	8

1 Introduzione

1.1 Scopo del Documento

Il presente documento raccoglie i risultati dell'analisi preliminare delle tecnologie considerate per la realizzazione del Proof of Concept del progetto *Second Brain*, proposto da Zucchetti S.p.A. nel capitolato C6.

L'obiettivo del documento non è definire l'architettura definitiva del prodotto, che verrà consolidata nella successiva fase PB, ma motivare le scelte tecnologiche adottate per la Technology Baseline e individuare i principali rischi tecnici da validare tramite il PoC.

1.2 Scopo del Prodotto

Il prodotto sviluppato consiste in una piattaforma documentale intelligente basata su tecnologie AI e Large Language Models (LLM_G), finalizzata al supporto della scrittura e della gestione di note Markdown_G.

La piattaforma dovrà supportare funzionalità di:

- gestione documentale Markdown;
- supporto AI alla scrittura;
- generazione di suggerimenti contestuali;
- interazione con Large Language Models;
- gestione strutturata delle note;
- interfaccia utente_G moderna, accessibile e responsive.

1.3 Aree di Indagine

L'analisi verte sulle seguenti aree di sviluppo.

- Tecnologie frontend: valutazione di framework e librerie per interfacce reattive.
- Tecnologie backend: analisi di framework per API RESTful e gestione logica.
- Struttura tecnica preliminare: definizione dell'organizzazione del PoC e gestione dell'ambiente di esecuzione.
- Sicurezza e protezione dei dati: strategie per la protezione delle comunicazioni e dei dati sensibili.

1.4 Glossario

I termini tecnici, gli acronimi e le espressioni potenzialmente ambigue utilizzate all'interno della documentazione sono definiti nel documento [Glossario](#).

La prima occorrenza dei termini presenti nel glossario viene marcata automaticamente tramite pedice_G mediante uno script Python dedicato descritto nella Sezione ??.

1.5 Riferimenti

1.5.1 Normativi

- Capitolato d'appalto C6: [Zucchetti S.p.A.](#)
- [Regolamento del Progetto Didattico](#), Prof. Tullio Vardanega, A.A. 2025/2026.
- [Norme di Progetto](#)

1.5.2 Informativi

- Documentazione ufficiale Vue.js: [Vue.js Documentation](#);
- Documentazione ufficiale CodeMirror: [CodeMirror Documentation](#);
- Documentazione ufficiale FastAPI: [FastAPI Documentation](#);
- Documentazione ufficiale PostgreSQL: [PostgreSQL Documentation](#);
- Documentazione ufficiale Docker: [Docker Documentation](#).

2 Tecnologie Frontend

Il gruppo ha condotto un'analisi comparativa sui principali framework e librerie JavaScript / TypeScript attualmente leader del settore. L'obiettivo era individuare lo strumento più idoneo per lo sviluppo di un'interfaccia reattiva, capace di gestire la necessità di mantenere il sistema semplice e intuitivo anche per utenti non esperti.

Sono state analizzate le seguenti alternative.

- **Angular:** framework vastamente utilizzato e completo.
 - *Pro:* offre una struttura robusta e strumenti integrati (routing, gestione stati, client HTTP) senza dipendere da librerie terze.
 - *Contro:* curva di apprendimento estremamente ripida, è stato ritenuto eccessivamente pesante per gli obiettivi del sistema richiesto da Zucchetti.
- **React:** principale alternativa ad Angular per diffusione e integrazione.
 - *Pro:* ecosistema maturo, performance ottimizzate, ampia disponibilità di librerie e pattern consolidati.
 - *Contro:* richiede la definizione preventiva di standard architetturali (state management, routing, styling) per evitare frammentazione nel team. Data l'assenza di esperienza pregressa con framework moderni, è stato valutato come ad alto rischio di debito tecnico iniziale.
- **Vue.js:** tecnologia scelta dal gruppo per lo sviluppo frontend.
 - *Pro:* progressività (inizio da codice semplice e intuitivo, man mano si aggiunge complessità ove necessario), separazione di struttura HTML e styling CSS, ottimizzato per granularità della User Interface.
 - *Contro:* ecosistema leggermente meno vasto di React in ambito enterprise, ma pienamente sufficiente per le funzionalità richieste dal capitolato.

Inoltre, nelle tecnologie frontend ricade la scelta di un altro componente critico: l'editor di testo. Le alternative analizzate in questo caso sono state due.

- **Monaco Editor:** motore alla base dell'editor di testo VS Code, potente e completo ma con elevato consumo di risorse ed una complessa configurazione.
- **CodeMirror:** componente scelto dal gruppo per bilanciamento tra leggerezza e estensibilità. Ideale per l'implementazione del sistema Second Brain, fornisce la possibilità di integrare comandi personalizzati per l'interazione con il LLM e/o altre feature del sistema, ha un ecosistema di estensioni pronte all'uso per Markdown, syntax highlighting, keybindings, etc.

Sono stati presi in considerazione anche altri editor, come TipTap e Milkdown, tuttavia sono stati scartati per la complessità di configurazione, inoltre richiedono maggiore esperienza per mantenere la coerenza tra modello interno e markdown generato.

3 Tecnologie Backend

La scelta dell'ecosistema backend è stata guidata da tre requisiti critici: la necessità di elaborare dati testuali tramite modelli di intelligenza artificiale (LLM), la velocità di risposta delle API e

la facilità di integrazione con il database relazionale.

Come linguaggio backend è stato scelto Python, principalmente per la disponibilità di un ecosistema maturo per l'integrazione con strumenti di Intelligenza Artificiale, NLP e Large Language Models.

La conoscenza pregressa del linguaggio da parte di alcuni membri del gruppo ha rappresentato un ulteriore fattore favorevole, riducendo il rischio di apprendimento iniziale e consentendo una più rapida realizzazione del PoC.

Per quanto riguarda, invece, i framework di gestione delle api, sono state valutate le seguenti alternative.

- **Django:** soluzione completa per applicazioni web e API in Python.
 - *Pro:* soluzione pronta all'uso su tutti i fronti, sicurezza integrata di alto livello.
 - *Contro:* eccessivamente pesante e rigido per un sistema semplice e dove la rapidità è chiave, la sua natura sincrona lo rende meno adatto a lavori di parallelizzazione.
- **Flask:** estremamente leggero.
 - *Pro:* semplice da configurare, ideale per piccoli prototipi.
 - *Contro:* manca di strumenti all'avanguardia, validazione dei dati o supporto asincrono, richiede l'uso di "plugin" esterni per raggiungere le funzionalità minime richieste.
- **FastAPI:** tecnologia scelta dal gruppo.
 - *Pro:* elevate prestazioni, pensato per chiamate asincrone (fondamentali nelle chiamate al LLM), supporta la validazione dei dati, genera la documentazione in modo automatico utilizzando Swagger, permette facile gestione di middlewares per logging e sicurezza, integrazione di rate limiting (protezione delle API dei LLM).
 - *Contro:* curva di apprendimento leggermente più complessa rispetto ad alternative basilari come Flask.

La gestione dei dati (salvataggio di note), pur essendo un requisito opzionale, richiede la valutazione di un sistema di database.

Durante l'analisi, il gruppo ha analizzato sia un approccio SQL, sia un approccio NoSQL. Rispettivamente, i motori selezionati sarebbero stati PostgreSQL (preferito dalla proponente Zucchetti) e MongoDB (flessibile nel gestire documenti non strutturati).

La scelta finale è però ricaduta su **PostgreSQL**.

- Sistema relazionale più solido rispetto ad un sistema NoSQL.
- Affidabile gestione delle transazioni.
- Indexing avanzato per ricerche full-text.
- Licenza open source, a differenza della licenza più restrittiva di MongoDB.
- Supporto al salvataggio di informazioni JSON, offrendo la flessibilità dei database non relazionali pur mantenendo la robustezza di un sistema SQL.
- Possibilità di valutare estensioni per la gestione di embeddings, qualora nelle fasi successive emerga l'esigenza di rappresentare relazioni semantiche tra le note.

Tale scelta non è stata validata nel PoC, poiché la persistenza non rientra tra i rischi tecnologici principali della RTB. PostgreSQL viene quindi indicato come tecnologia candidata per le successive fasi progettuali.

4 Struttura Tecnica Preliminare del Proof of Concept

In fase RTB il gruppo non ha l'obiettivo di definire l'architettura definitiva del prodotto. La progettazione architetturale completa verrà affrontata nella fase PB, sulla base dei risultati del Proof of Concept, dei requisiti consolidati e delle successive attività di progettazione.

Per la Technology Baseline è stata quindi individuata una struttura tecnica preliminare, limitata al perimetro del PoC, con lo scopo di validare l'integrazione tra le principali componenti tecnologiche.

Il PoC è organizzato secondo una struttura client-server composta da:

- **Frontend:** applicazione Vue.js responsabile dell'interfaccia utente, dell'editor Markdown, dell'anteprima e della gestione delle interazioni dell'utente;
- **Backend:** servizio FastAPI responsabile dell'esposizione delle API REST, della ricezione delle richieste provenienti dal frontend e dell'interazione con il servizio LLM;
- **Servizio LLM esterno:** componente esterno utilizzato per generare proposte testuali a partire dal contenuto selezionato o dal prompt fornito dall'utente.

Nel PoC non è stato incluso un database, poiché la persistenza dei dati non rappresentava uno dei principali rischi tecnologici da validare in RTB. La sua introduzione verrà rivalutata nella fase PB.

È importante evidenziare che la separazione tra frontend e backend, e la loro esecuzione in container distinti, non costituisce una scelta architetturale definitiva: i container sono utilizzati in questa fase esclusivamente per rendere l'ambiente di esecuzione più riproducibile, portabile e semplice da avviare.

5 Distribuzione dell'Ambiente

Per la distribuzione dell'ambiente di sviluppo e dimostrazione sono state considerate tre alternative principali:

- **Installazione manuale:** semplice da comprendere, ma poco controllabile, poiché richiede a ogni membro del gruppo di configurare manualmente runtime, dipendenze e variabili d'ambiente;
- **Macchina virtuale:** garantisce isolamento completo dell'ambiente, ma introduce un overhead superiore rispetto alle esigenze del PoC;
- **Docker Compose:** consente di descrivere frontend e backend come servizi separati e di avviarli tramite un unico comando.

La scelta è ricaduta su Docker Compose, poiché permette di ridurre le differenze tra ambienti locali, semplificare l'esecuzione del PoC e rendere più agevole la dimostrazione della Technology Baseline.

6 Sicurezza e Protezione dei Dati

L'analisi delle tecnologie ha evidenziato che il dominio applicativo di *Second Brain* può coinvolgere informazioni personali, note private, contenuti sensibili e proprietà intellettuale dell'utente. Inoltre, alcune funzionalità previste richiedono l'invio di porzioni di testo a servizi LLM esterni.

In fase RTB, il PoC non ha l'obiettivo di implementare in modo completo i meccanismi di sicurezza del prodotto finale. Le considerazioni riportate in questa sezione rappresentano quindi una presa di consapevolezza dei principali aspetti di sicurezza da considerare nelle fasi successive, compatibilmente con i vincoli di tempo, competenze e risorse disponibili.

Le principali aree di attenzione individuate sono:

- **Protezione dei dati in transito:** in un prodotto completo, le comunicazioni tra frontend, backend ed eventuali servizi esterni dovrebbero essere protette tramite protocollo HTTPS e cifratura TLS, al fine di ridurre il rischio di intercettazione dei dati;
- **Gestione delle credenziali:** le API key per l'accesso ai servizi LLM e, nelle fasi successive, le eventuali stringhe di connessione a basi di dati non dovrebbero essere inserite direttamente nel codice sorgente. Nel PoC questa attenzione è stata recepita tramite l'uso di variabili d'ambiente, quando applicabile;
- **Autenticazione e autorizzazione:** qualora il sistema evolva verso una gestione multiutente, sarà necessario valutare meccanismi di autenticazione e controllo degli accessi, ad esempio basati su token. Tali aspetti non sono stati implementati nel PoC, poiché non rientravano tra i rischi tecnologici prioritari della RTB;
- **Tutela dei contenuti inviati agli LLM:** poiché alcune funzionalità prevedono l'invio di testo a modelli esterni, sarà opportuno valutare strategie per limitare l'esposizione di informazioni personali o sensibili. Una possibile direzione è introdurre, nelle fasi successive, controlli o avvisi all'utente prima dell'invio del contenuto;
- **Controllo dell'uso dei servizi LLM:** l'uso di servizi esterni può comportare limiti di utilizzo, costi e rischi di abuso. In una versione più matura del prodotto, potranno essere valutati meccanismi di rate limiting o controllo delle richieste.

Le misure sopra indicate rappresentano linee guida tecniche da considerare durante la progettazione e lo sviluppo delle fasi successive, in base alla priorità dei requisiti e alla disponibilità di risorse.

7 Scelte Finali

L'analisi approfondita dello Stato dell'Arte ha permesso al gruppo Synergo Unit di definire uno stack tecnologico solido, moderno e strettamente mirato a soddisfare i requisiti del capitolato C6 proposto da Zucchetti.

La sinergia tra le tecnologie selezionate definisce la seguente struttura tecnica di riferimento per il PoC.

- **Frontend (Vue.js + CodeMirror):** offre un'interfaccia utente (UI) reattiva, leggera e caratterizzata da una bassa curva di apprendimento. L'integrazione di CodeMirror garantisce la flessibilità necessaria per manipolare il testo in formato Markdown, permettendo lo sviluppo di plug-in personalizzati per comunicare con l'LLM direttamente dalla finestra di scrittura.

- **Backend (Python + FastAPI)**: fornisce un ecosistema ad alte prestazioni e nativamente asincrono. Questa caratteristica è fondamentale per gestire i tempi di latenza tipici delle risposte degli LLM (per riscritture, riassunti o traduzioni) senza bloccare l'esperienza d'uso dell'utente.
- **Database (PostgreSQL)**: pur trattandosi di un requisito opzionale e non incluso nel PoC, PostgreSQL è stato individuato come tecnologia candidata per le fasi successive. La scelta è motivata dalla natura relazionale dei dati previsti, quali utenti, note, metadati e relazioni tra entità, oltre che dal supporto a transazioni e vincoli di integrità. Eventuali estensioni per la gestione di embeddings potranno essere valutate in PB, qualora emergano esigenze legate alla rappresentazione semantica delle note.
- **Deployment (Docker Compose)**: la containerizzazione dei componenti del PoC garantisce portabilità, isolamento delle dipendenze e semplicità di esecuzione. Docker Compose è stato utilizzato per avviare frontend e backend in modo coordinato, senza vincolare il gruppo a una specifica architettura definitiva.

In conclusione, il gruppo ritiene che questo stack rappresenti il compromesso ideale tra velocità di sviluppo e solidità tecnica.

La soluzione così delineata permetterà di validare l'efficacia del sistema di **Second Brain**, fornendo una base tecnica per l'evoluzione del note-taking verso funzionalità di co-authoring e distant writing descritte dalla proponente.